

Retrieval recall $\neq$ LLM Recall

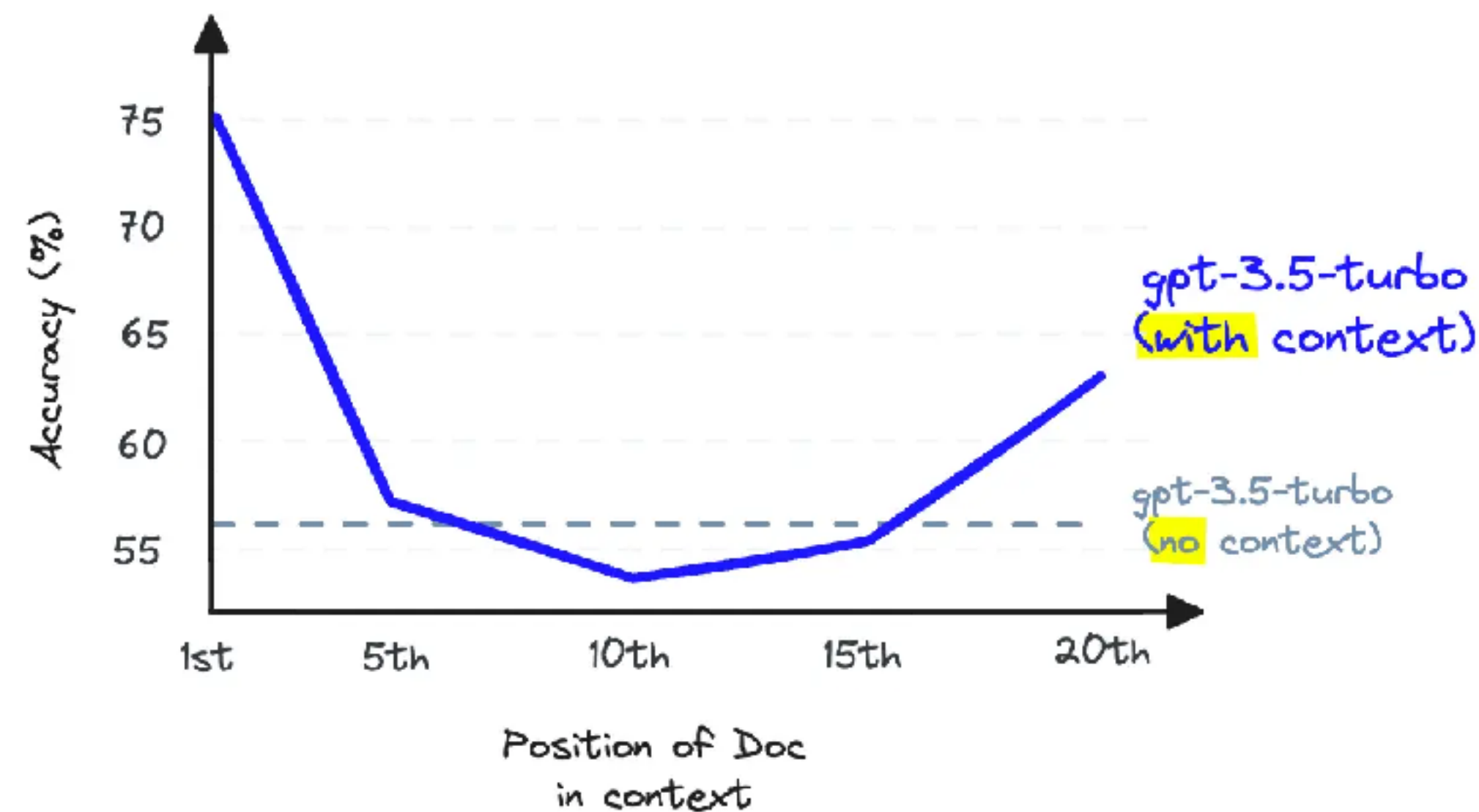
Retrieval recall: Does the retriever bring back all relevant documents?

LLM recall: Once given documents, does the LLM actually use them?

Example:

Retriever returns 5 docs, but only 3 are relevant. LLM might miss relevant facts even if from the 3 relevant docs that are present.

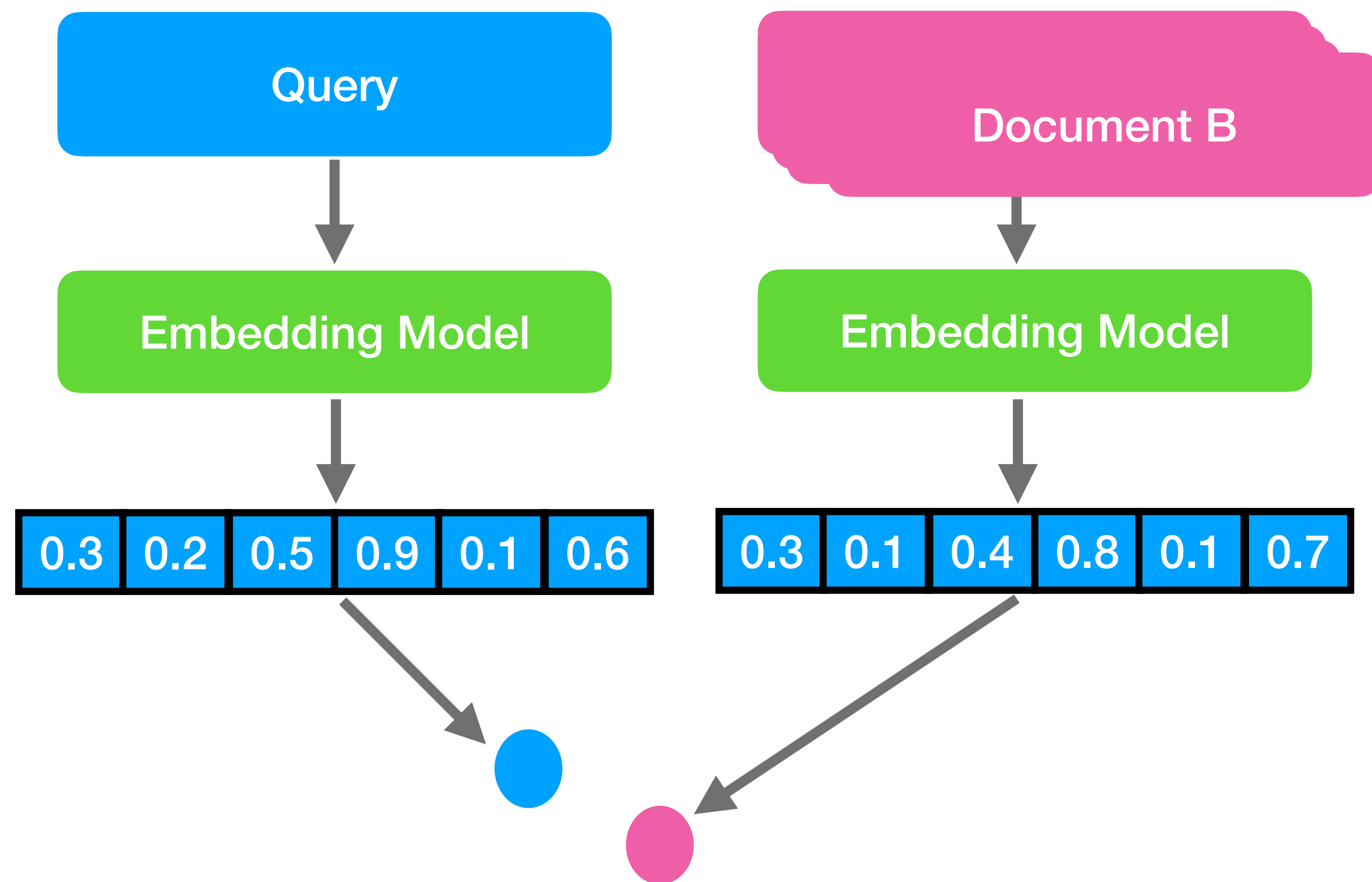
Why would LLM not recall ?



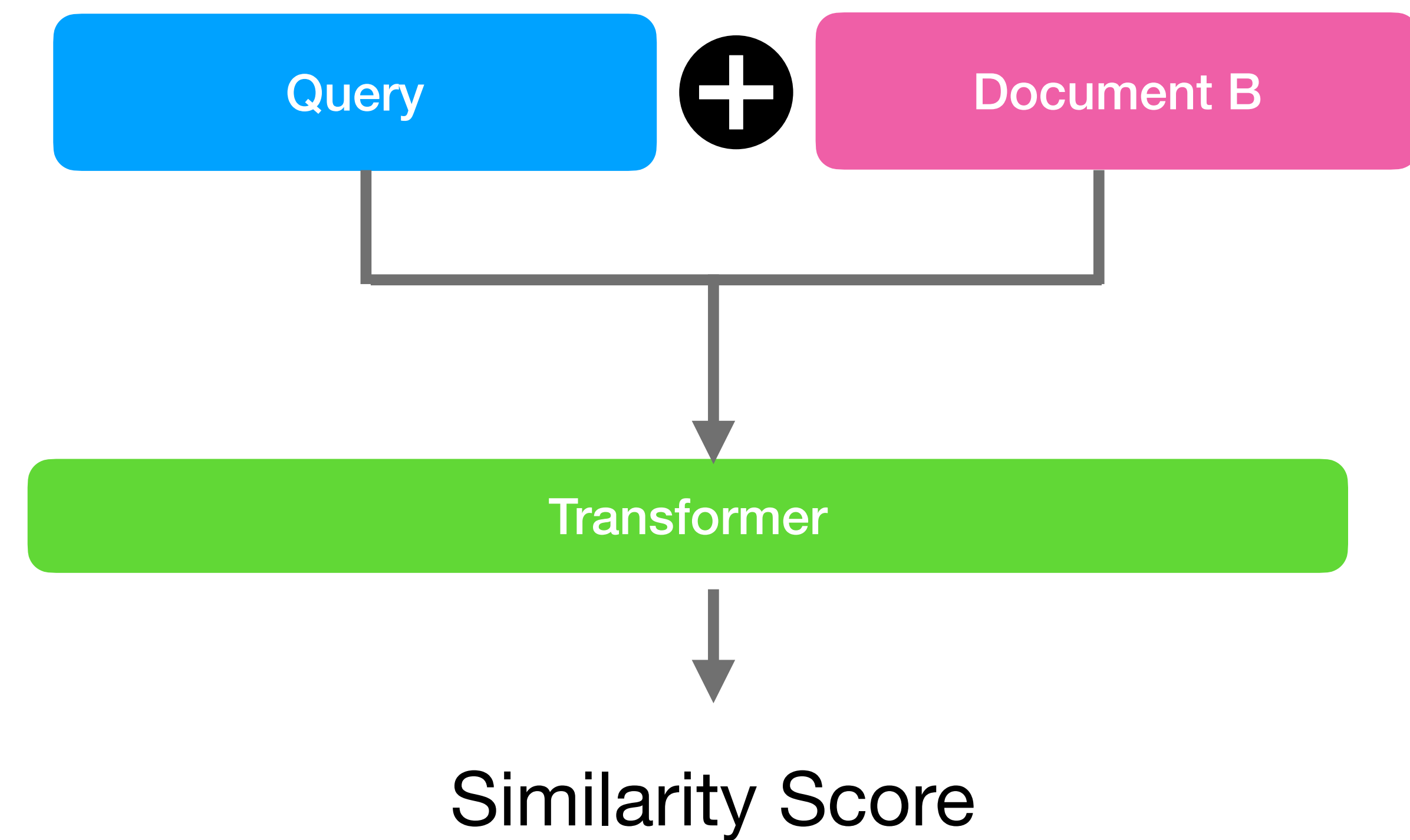
- LLMs pay less attention to tokens in the middle of the context window.
- **Evidence:** Studies show models bias toward **first tokens (primacy)** and **last tokens (recency)**.
- **Implication in RAG:** If relevant docs are in the middle of prompt, model may underuse them.

Two ways to retrieve

Bi-Encoder



Cross-Encoder



Calculate proximity to get similarity

Bi-Encoder Cross Encoder Tradeoffs

Bi-Encoder

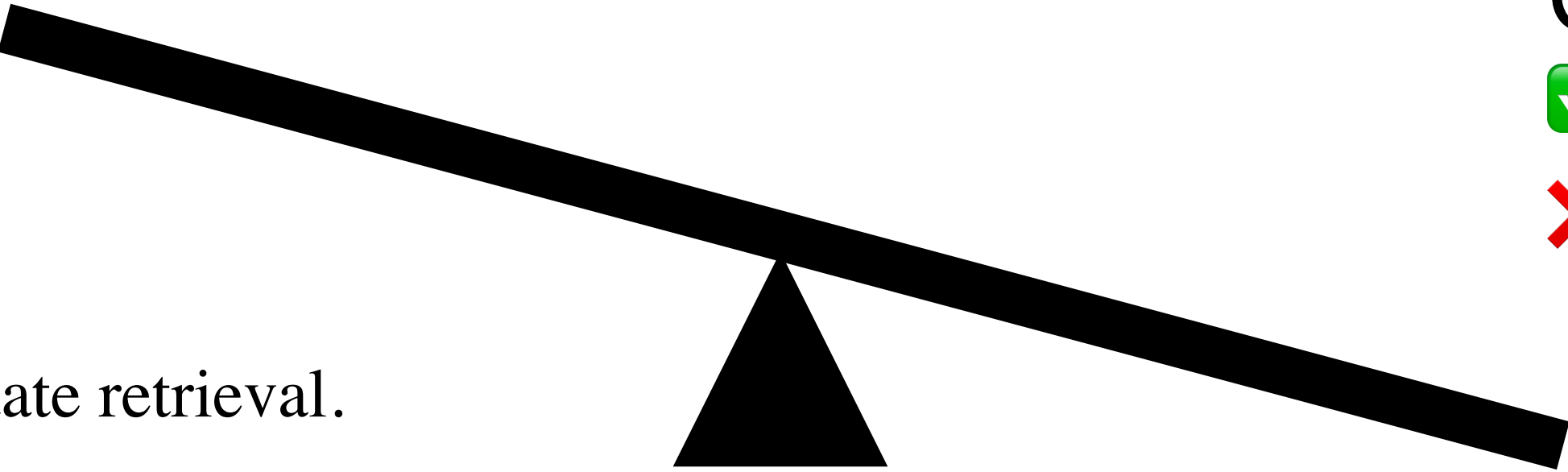
✓ Fast, scalable → used for candidate retrieval.

✗ Sometimes poor at fine-grained distinctions.

Cross-Encoder

✓ More accurate semantic matching.

✗ Too slow for millions of docs.

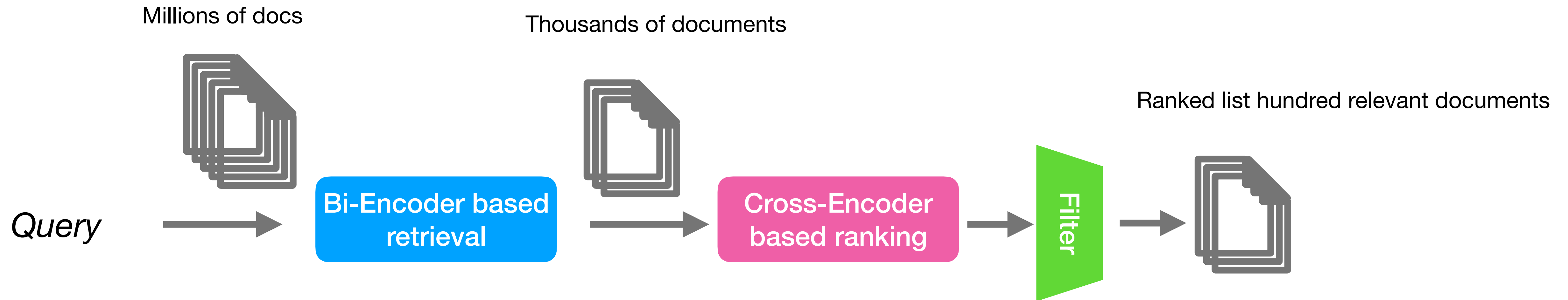


Can we get the best of both worlds?

Comparison Between Bi-Encoder and Cross-Encoder

Aspect	Bi-Encoder (Dual Encoder)	Cross-Encoder
Architecture	Two separate, identical encoders	Single encoder processes both inputs together
Input Processing	Query and candidate encoded independently	Query and candidate concatenated & encoded jointly
Output	Separate embeddings (vector representations)	Direct similarity or relevance score
Comparison Method	Cosine similarity or dot product of embeddings	Learned attention across both inputs
Speed & Scalability	Very fast – embeddings can be pre-computed & indexed	Slow – must process each pair at inference
Typical Use Case	Retrieval (search over thousands of candidates)	Re-ranking (scoring top ~10–100 candidates)
Accuracy	Good, but may lose fine-grained interaction details	Higher accuracy due to deep interaction modeling
Example Usecase	Finding relevant documents in a large corpus	Ranking the top 10 search results precisely

Two stage Retrieval



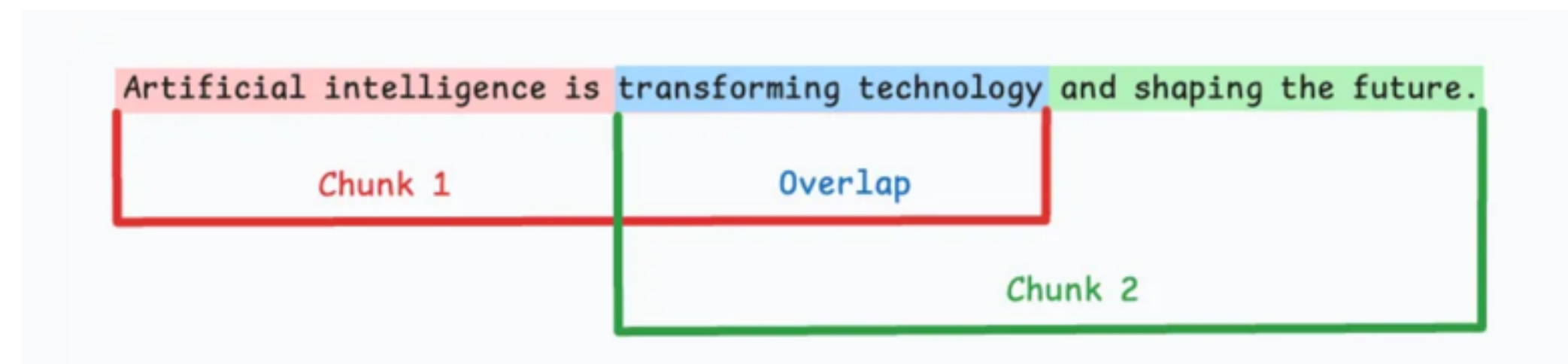
What is my chunking strategy?

- Chunking is the process of breaking down a large document into smaller, more manageable pieces called **chunks**.
- RAG retrieves only the most relevant chunks instead of entire documents, ensuring efficiency, lower cost, and better answer quality.
- There's a key **tradeoff** between **recall** and **precision**.
- **Large chunks** => high recall but low precision
- **Small chunks** => low recall but high precision

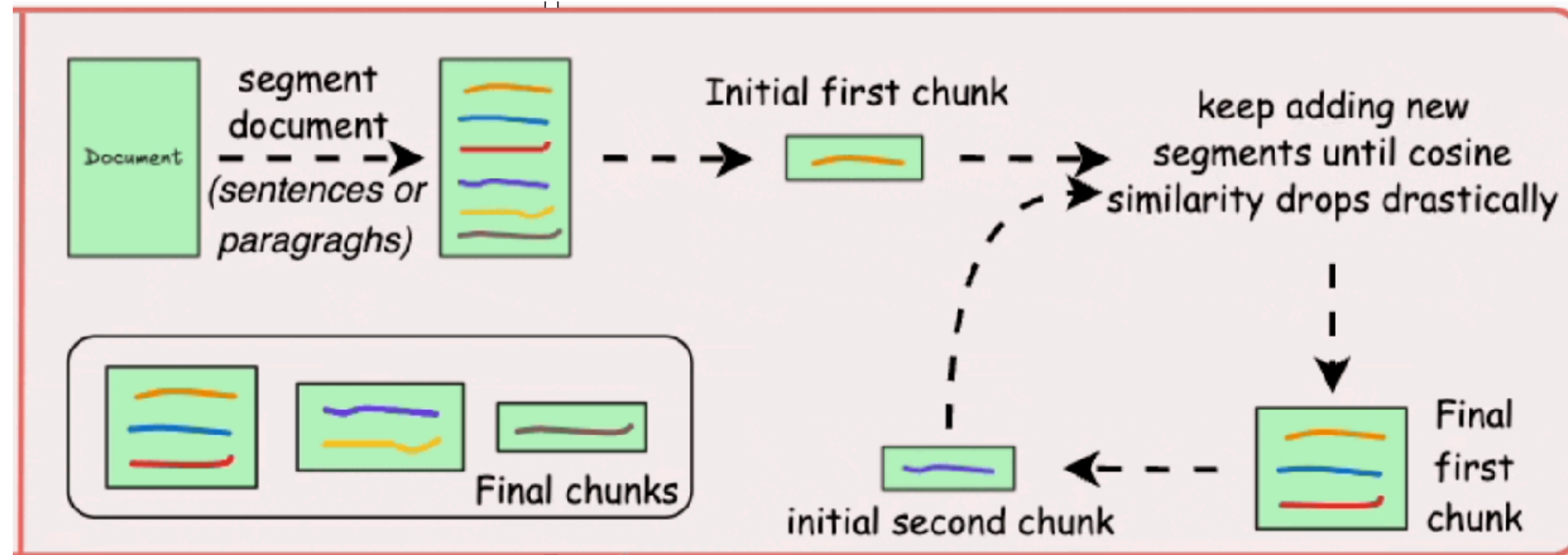
Chunking strategies: Fixed Size Chunking

Split a document into chunks of a **fixed size**, based on a set number of **characters** or **tokens**.

- **Advantage:** It's **simple** to implement and **fast** to execute. Good starting point
- **Disadvantage:** The main drawback is that it can **cut off semantic meaning** mid-sentence or mid-paragraph.



Chunking strategies: Semantic Chunking

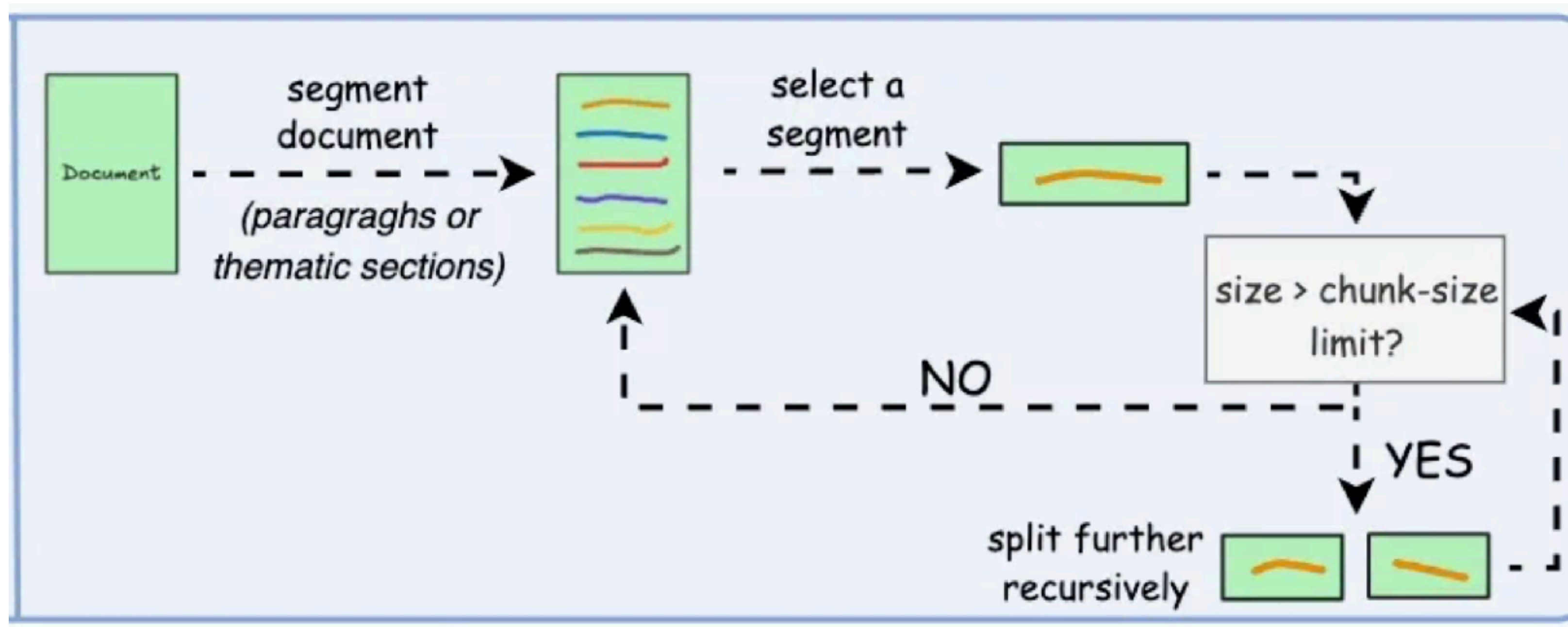


Artificial intelligence is transforming industries by automating processes, enhancing decision-making, and providing insights through data analysis. Machine learning, a subset of AI, enables systems to learn and improve from experience without explicit programming. Deep learning, a branch of machine learning, uses neural networks with multiple layers to model complex patterns in data.

Advantage: Retrieval accuracy goes up because of richer more meaningful chunks

Disadvantage: Choosing the threshold for similarity can be tricky

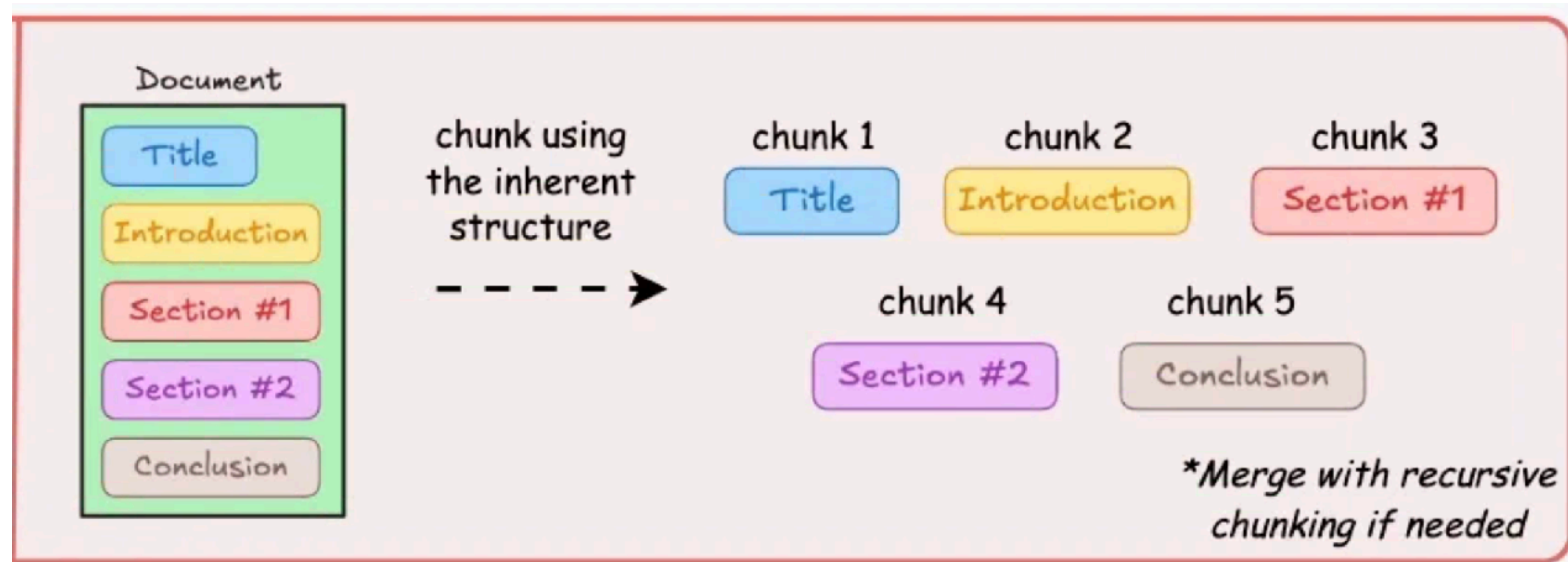
Chunking strategies: Recursive Chunking



Advantage: Retrieval accuracy goes up because of richer more meaningful chunks

Disadvantage: Implementation and computational complexity

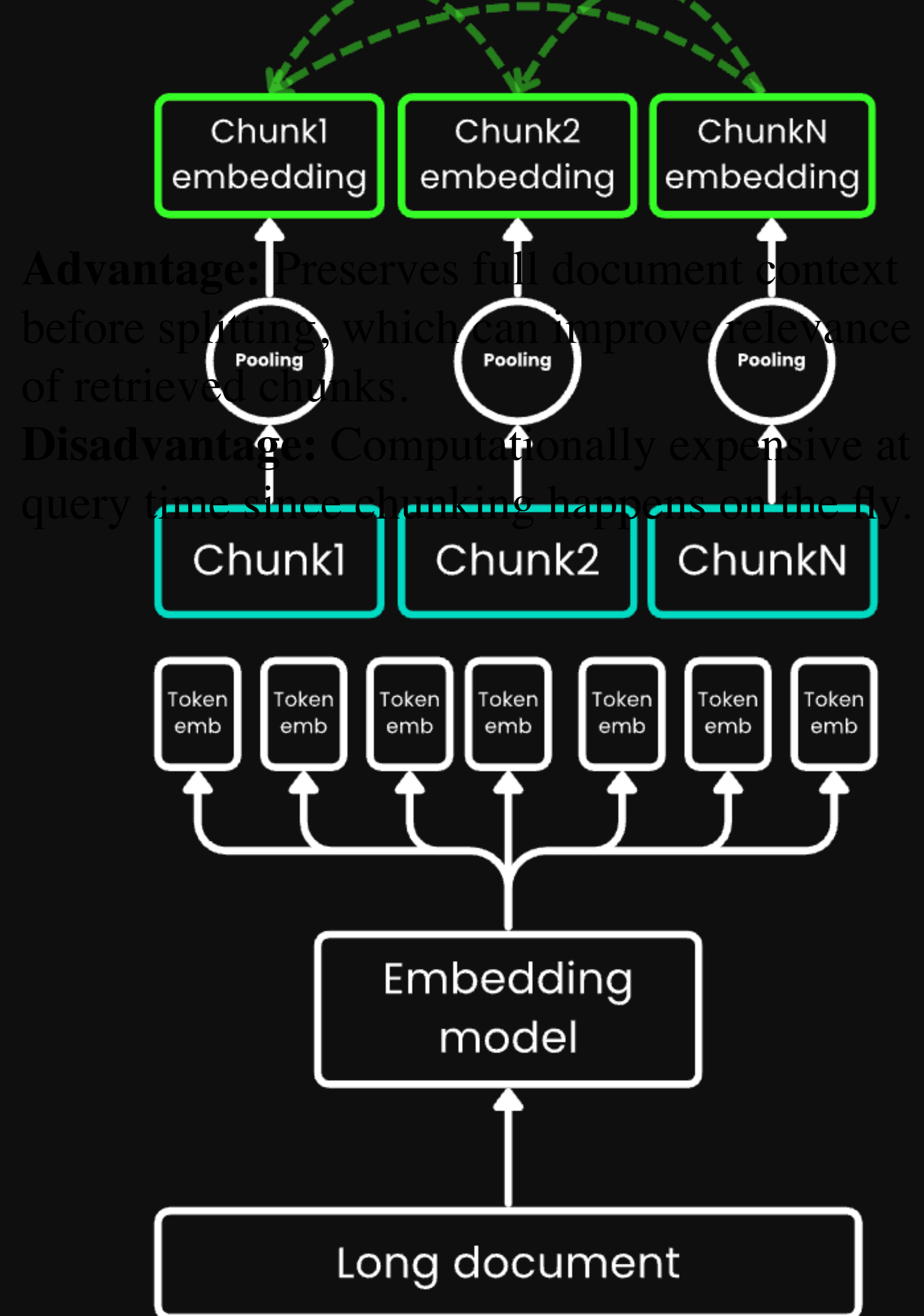
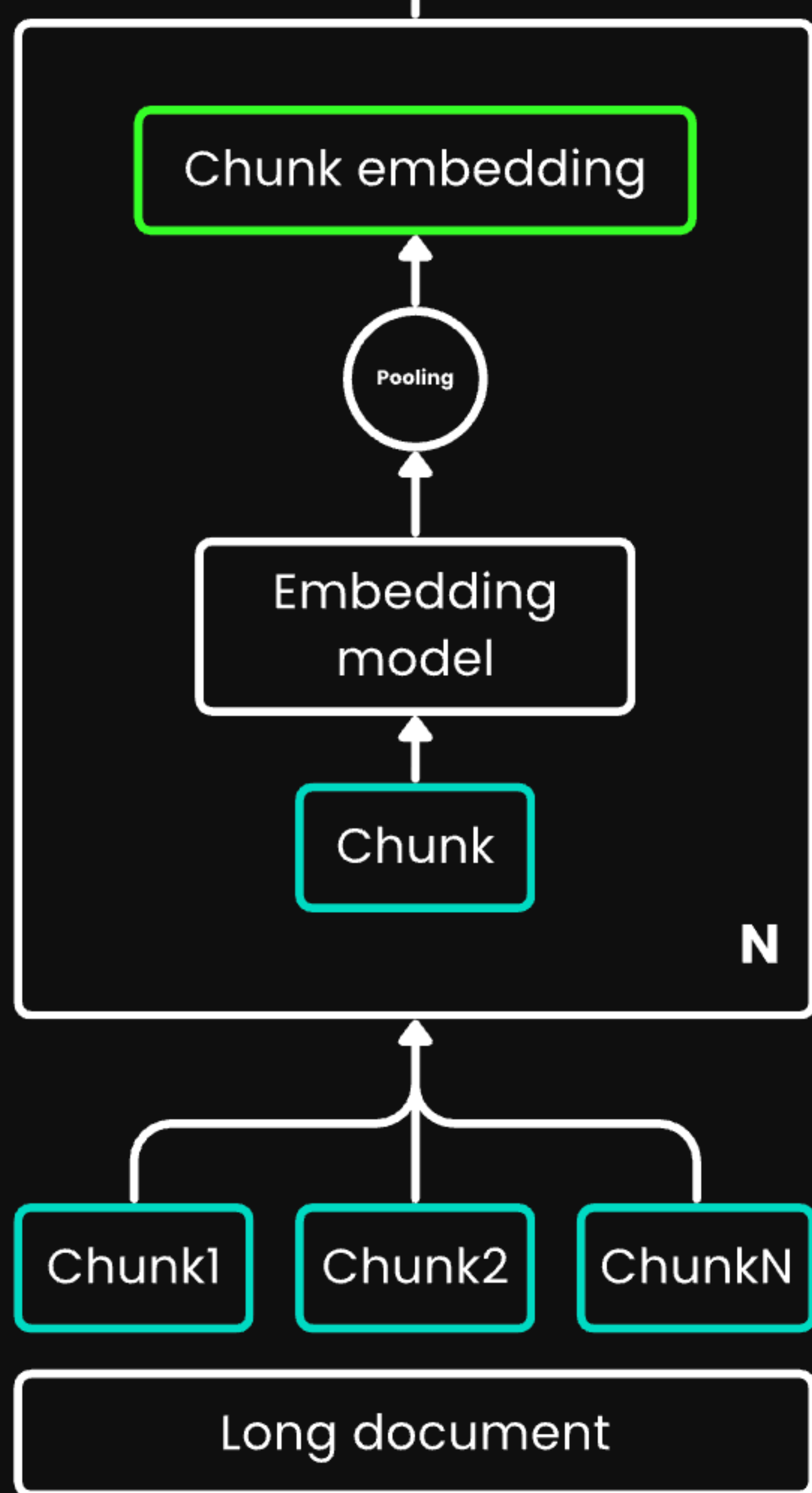
Chunking strategies : Document structure based chunking



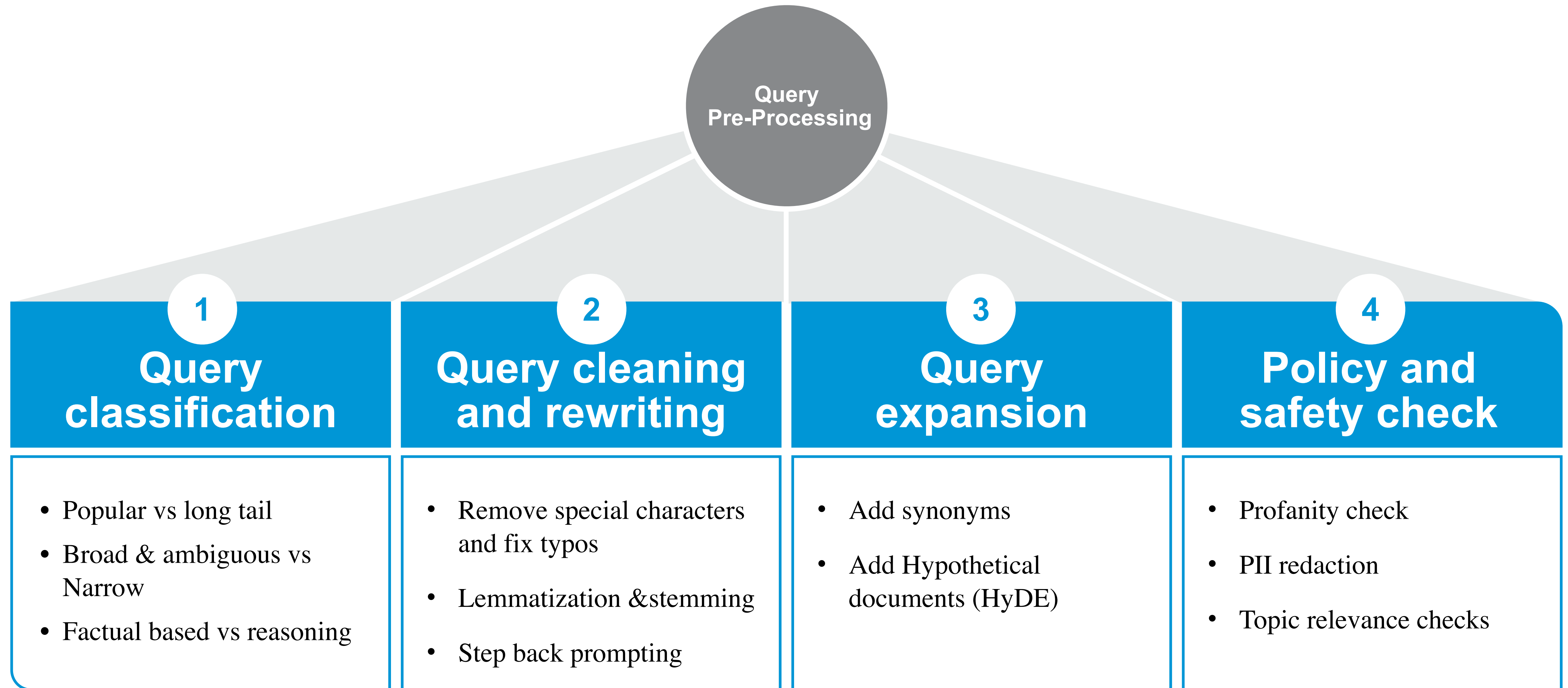
Title: The Role of Artificial Intelligence in Modern Education	Chunk 1
Introduction	
Artificial intelligence (AI) is reshaping education by providing personalized learning experiences and automating administrative tasks.	Chunk 2
Section 1: Personalized Learning	
AI enables the customization of educational content to meet individual student needs, enhancing engagement and comprehension.	Chunk 3
Section 2: Administrative Automation	
From grading to scheduling, AI tools are streamlining administrative processes, allowing educators to focus more on teaching.	Chunk 4

Advantage: Retrieval accuracy goes up because document structure is preserved

Disadvantage: Might have to be clubbed with recursive text splitting because each token may be too large



Query pre-processing - Modify queries to improve retrieval



Choosing the right embedding model in RAG



RELEVANCE: THE PRIMARY FACTOR IS HOW ACCURATELY THE MODEL MAPS TEXT TO A VECTOR SPACE. FOR RAG, THE MODEL MUST EXCEL AT RETRIEVAL



DOMAIN SPECIFICITY: A GENERAL-PURPOSE EMBEDDING MODEL MIGHT NOT BE OPTIMAL FOR SPECIALIZED DOMAINS LIKE LAW, MEDICINE, OR FINANCE.



PERFORMANCE VS. COST: BALANCE MODEL QUALITY WITH PRACTICAL CONSTRAINTS. LARGER, MORE POWERFUL MODELS MAY OFFER SUPERIOR PERFORMANCE BUT COME WITH HIGHER COMPUTATIONAL COSTS FOR BOTH STORAGE (VECTOR DIMENSIONS) AND INFERENCE (LATENCY).



LANGUAGE SUPPORT: IF YOUR APPLICATION NEEDS TO HANDLE MULTIPLE LANGUAGES, YOU MUST SELECT A MULTILINGUAL MODEL THAT PERFORMS WELL ACROSS ALL TARGET LANGUAGES.



HOSTING: CONSIDER IF YOU'LL USE AN API-BASED MODEL (E.G., OPENAI, COHERE) FOR CONVENIENCE OR HOST AN OPEN-SOURCE MODEL LOCALLY FOR MORE CONTROL AND DATA PRIVACY.

The Massive Text Embedding Benchmark (MTEB)

- The Massive Text Embedding Benchmark (MTEB) is a crucial resource for evaluating and comparing the performance of embedding models. Excellent starting point
- **What it is:** A comprehensive, standardized benchmark that evaluates embedding models across a diverse set of tasks and datasets.
- **What it measures:** MTEB provides scores for a model's performance on eight different tasks, including retrieval, semantic textual similarity (STS), classification, and clustering. This holistic approach shows how well a model generalizes.
- **How to use it:** You should look at the retrieval scores on the MTEB leaderboard as a primary indicator of a model's suitability for RAG.
- Don't rely solely on the overall average score. Instead, examine the scores on specific datasets that are most relevant to your application's domain and use case.

Quiz 1: Choose the best embedding model for the Legal Assistant

You are building a RAG system for a legal firm to search through case law and complex legal documents. The top priority is high accuracy in finding relevant legal precedents from a massive, specialized database.

Which embedding model is likely the BEST choice?

- A) text-embedding-ada-002
- B) BAAI/bge-large-en-v1.5
- C) thenlper/gte-large
- D) lexlms/legal-bert
- E) intfloat/multilingual-e5-large

Quiz 2: Choose the best embedding model for the Global News Aggregator

You are developing a news search engine that must retrieve articles in English, Spanish, French, and Mandarin Chinese. Performance must be consistent across all these languages.

Which factor is MOST critical and which model aligns with it?

- A) BAAI/bge-large-en-v1.5
- B) Choose a model fine-tuned on news data.
- C) sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2.
- D) all-MiniLM-L6-v2
- E) Any other model from MTEB

Which vector database to choose?

Parameter	Pinecone	Chroma	Qdrant
Type	Fully managed SaaS	Open-source Self-hosted and cloud options	Open-source with self-hosted and cloud options
Scalability	Massive scale Handles billions of vectors automatically with no DevOps effort.	Prototyping/small scale Ideal for up to 10M vectors	Massive scale. Built to handle billions of vectors.
Ease of use	Easy. Abstracts away infra complexity	Simple and Python-native. Easy to get started with for R&D	Moderate. Slightly more complex than ChromaDB due to its production features, but provides powerful tools for cluster management and
Performance	Low latency high throughput	Good for small scale	Low latency high throughput
Cost	Tiered pricing. Priced based on usage. Can become expensive at a large scale.	Free and open-source. Costs are limited to your own infrastructure.	Free and open-source. Costs are limited to your own infrastructure or a managed cloud service

Evaluating RAG - Relevance

- **What it measures:** How well the retrieved information matches the user's question.
- **Focus:** Query-to-retrieval alignment.
- Does the retrieved context actually relate to what was asked?
- Are the retrieved documents/chunks topically relevant?
- **Example:**
 - **Question:** "What causes lightning?"
 - **Relevant retrieval:** Documents about electrical discharge, thunderstorms.
 - **Irrelevant retrieval:** Documents about car batteries, house wiring.

Evaluating RAG - Faithfulness

What it measures: How well the generated answer sticks to the retrieved information.

Focus: Retrieved-context-to-answer alignment.

Does the answer only use information from the retrieved context?

Are there hallucinations or made-up facts not in the source?

Example:

- **Retrieved context:** "Lightning is caused by electrical discharge between clouds."
- **Faithful answer:** "Lightning occurs due to electrical discharge between clouds."
- **Unfaithful answer:** "Lightning is caused by aliens and happens every Tuesday." (hallucinated content)

Evaluating RAG - Correctness

What it measures: How factually accurate the final answer is.

Focus: Answer-to-ground-truth alignment.

Is the information in the answer actually true?

Does it match verified facts or expert knowledge?

Example:

- **Question:** "What's the speed of light?"
- **Correct:** "299,792,458 meters per second."
- **Incorrect:** "150,000 km/s." (wrong number)

Summary: Key design questions to ask while building a RAG system

1

Query preprocessing

2

Which embedding model to use?

3

What is my chunking strategy?

4

Which vector DB to choose?

5

What should be my retrieval strategy?

6

How to evaluate the RAG system?

Resources to read further

- **Chunking strategies:** <https://shorturl.at/YHrKe>
- **Embedding model Selection:** <https://huggingface.co/spaces/mteb/leaderboard>
- **RAG Evals:** <https://arxiv.org/pdf/2405.07437>
- **Vector DB Comparison:** <https://superlinked.com/vector-db-comparison>